



Universidade do Minho

Licenciatura em Engenharia Informática - LEI

Computação Gráfica

Trabalho Prático - Fase 2

Grupo 14

A107324 - David José Barbosa Alves

A107368 - Diogo Malheiro Pais

A107377 - João Pedro Ferreira Gomes

A107332 - Pedro Afonso Quesado Pacheco Pereira

1. Introdução

O presente relatório descreve o desenvolvimento da **segunda fase** do projeto prático da unidade curricular de Computação Gráfica. O objetivo central desta etapa é a evolução do motor de visualização para um sistema baseado num **Scene Graph** (grafo de cena) hierárquico, permitindo a gestão de cenas complexas através de transformações geométricas acumuladas.

Ao contrário da fase anterior, o trabalho realizado centrou-se exclusivamente na expansão das capacidades do **Engine**, mantendo-se a ferramenta **Generator** inalterada:

1. **Generator (Consolidação):** A aplicação de linha de comandos mantém a sua função de computação algorítmica das primitivas geométricas base (plano, cubo, esfera e cone). Os modelos gerados continuam a ser exportados para o formato proprietário .3d, que se revelou eficiente para o carregamento de dados na memória RAM através de vetores dinâmicos.
2. **Engine:** O motor gráfico foi significativamente atualizado para suportar a leitura de ficheiros XML hierárquicos. A arquitetura agora permite que os modelos sejam organizados em **grupos**, onde cada grupo pode conter as suas próprias transformações de **translação, rotação e escala**.

Nesta fase, a principal inovação reside no processamento recursivo da cena, onde os nós "filhos" herdam as transformações dos seus ascendentes. Esta estrutura serviu de base para a criação da cena de demonstração obrigatória: um modelo estático do **Sistema Solar**, com planetas e satélites posicionados de forma relativa e hierárquica.

2. Engine: Scene Graph e Gestão de Transformações

Nesta segunda fase, o motor gráfico evoluiu de um visualizador de modelos isolados para um sistema de **Scene Graph**. Esta arquitetura permite organizar a cena como uma árvore hierárquica, onde cada nó (grupo) pode conter transformações geométricas e modelos que influenciam todos os seus descendentes.

2.1. Parser XML Recursivo

A leitura do ficheiro de configuração passou a ser efetuada de forma recursiva utilizando a biblioteca **TinyXML-2**. Ao contrário da fase anterior, onde a leitura era linear, o motor agora percorre a estrutura de blocos <group>.

- **Algoritmo de Parsing:** Foi implementada a função `parseGrupo` que, ao detetar um novo grupo, processa sequencialmente as suas transformações (`translate`, `rotate`, `scale`). Se encontrar um bloco <models>, os ficheiros .3d são carregados através da função `loadModel`; se encontrar outro <group>, a função invoca-se a si própria (recursividade), garantindo que a hierarquia de profundidade é respeitada.

2.2. Gestão da Stack de Matrizes

Para implementar a herança de transformações (onde, por exemplo, a translação de um planeta afeta a posição da sua lua), utilizou-se a **Pilha de Matrizes** (*Stack*) do OpenGL através dos comandos `glPushMatrix` e `glPopMatrix`.

- **Isolamento de Transformações:** Antes de processar o conteúdo de um grupo, o estado atual da matriz de transformação é guardado com `glPushMatrix()`.
- **Aplicação Local:** As transformações definidas no XML para aquele grupo são aplicadas utilizando funções como `glTranslatef`, `glRotatef` e `glScalef`. Devido à natureza da pilha do OpenGL, estas transformações acumulam-se com as dos grupos superiores ("pais").
- **Desenho e Restauro:** Após o desenho dos modelos e dos subgrupos, utiliza-se `glPopMatrix()`, garantindo que as transformações daquele grupo não afetam os grupos "irmãos", mantendo a integridade da árvore de cena.

2.3. Configuração Dinâmica da Câmara

A câmara deixou de ter valores fixos no código para passar a ser definida integralmente no ficheiro XML. O motor processa os seguintes parâmetros:

- **Posição e LookAt:** Definem, respetivamente, o ponto de vista do observador e o alvo para o qual a câmara aponta.

- **Projeção:** Permite ajustar o FOV (*Field of View*) e os planos de corte (*near* e *far*), adaptando o volume de visualização à escala da cena.

2.4. Algoritmo de Renderização e Eficiência

A renderização é efetuada através de uma travessia **Depth-First** (procura em profundidade) na árvore de grupos.

- **Sistema de Coordenadas Local:** O uso da stack permite que cada grupo defina o seu próprio sistema de coordenadas local, herdando automaticamente as transformações dos seus antecessores.
- **Otimização de Carregamento:** Para maximizar a eficiência, os modelos e a estrutura da cena são carregados e interpretados apenas uma vez durante a inicialização (loadXML). Os dados são armazenados em estruturas `std::vector` na memória RAM, minimizando o processamento redundante durante o ciclo de desenho (*render loop*).

3. Demonstração: Sistema Solar Estático

Como prova de conceito das funcionalidades implementadas no *Engine*, foi desenvolvida uma cena representando o Sistema Solar. Esta demonstração serve como o teste definitivo à robustez do *Scene Graph* e à correta propagação de transformações geométricas.

3.1. Estruturação Hierárquica da Cena

A cena foi organizada seguindo a lógica de dependência gravitacional dos corpos celestes, traduzida para uma árvore de grupos no ficheiro `solar_system.xml`:

- **Nó Raiz (Sol):** O Sol ocupa o centro do sistema de coordenadas global. Todos os outros astros são descendentes deste grupo.
- **Grupos de Órbita (Planetas):** Cada planeta possui o seu próprio grupo. A posição de um planeta é definida por uma translação em relação ao Sol. Como estas esferas são "filhas" do grupo do Sol, qualquer transformação aplicada ao astro central afetaria todo o sistema (herança).
- **Subgrupos (Satélites):** A Lua, por exemplo, é definida dentro do grupo da Terra. Isto garante que a translação da Lua seja relativa à posição da Terra e não à do Sol, simplificando o cálculo das coordenadas.

3.2. Aplicação de Transformações Combinadas

Para atingir o realismo visual necessário nesta fase estática, foram combinadas as três transformações base:

1. **Translate:** Utilizada para distanciar os planetas do Sol e as luas dos respectivos planetas.
2. **Scale:** Fundamental para representar as enormes diferenças de magnitude entre os corpos celestes (ex: o raio do Sol vs o raio de Mercúrio).
3. **Rotate:** Embora a cena seja estática, a rotação foi utilizada para inclinar os eixos dos planetas e para posicionar os astros em diferentes pontos das suas órbitas teóricas, evitando que todos ficassem alinhados num único eixo.

3.3. Resultados Visuais

Através da configuração da câmara no XML (parâmetros position, lookAt e fov), é possível observar o sistema de diferentes perspectivas, validando o algoritmo de renderização *Depth-First*. Foi possível construir o modelo do sistema solar utilizando apenas uma primitiva base, através da aplicação sucessiva de diferentes transformações.

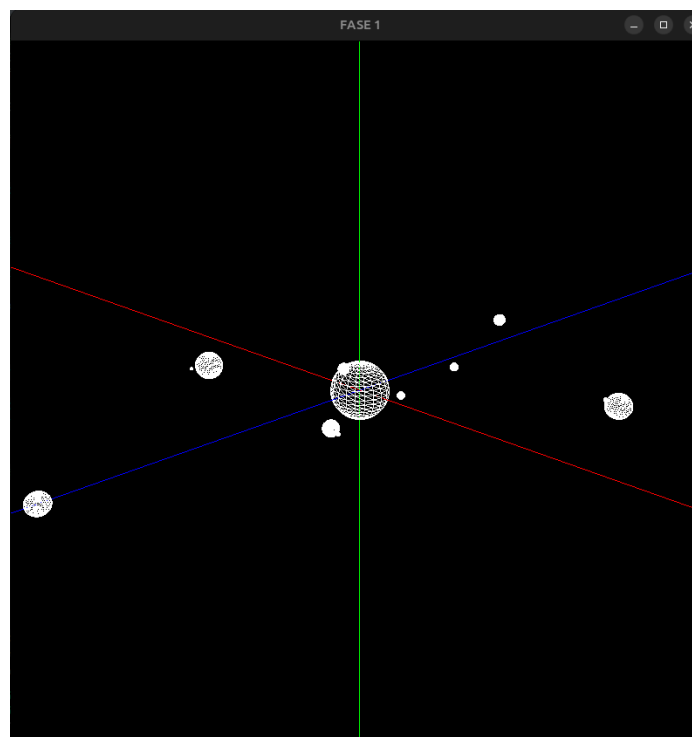


Figura 1: Sistema Solar Completo

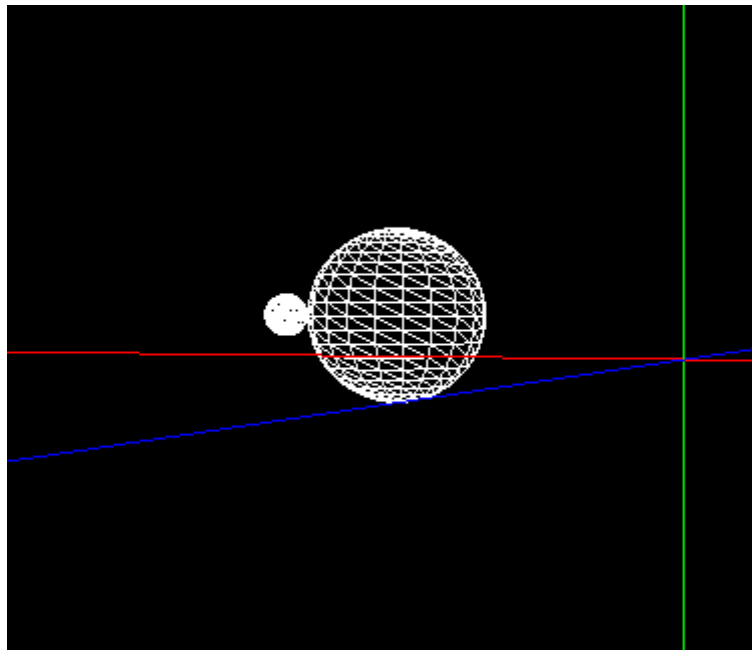


Figura 2: Detalhe Terra-Lua

4. Resultados e Testes

Nesta secção, apresenta-se a validação das ferramentas desenvolvidas através da execução dos ficheiros de configuração XML fornecidos pela equipa docente e da cena personalizada do Sistema Solar. O objetivo foi garantir que a geometria e as transformações hierárquicas fossem processadas corretamente pelo *engine*.

4.1. Validação das Cenas de Teste (Fase 2)

A correspondência exata entre as imagens de referência e a renderização do motor prova que a ordem de aplicação das transformações (Translação, Rotação e Escala) no código respeita a hierarquia definida.

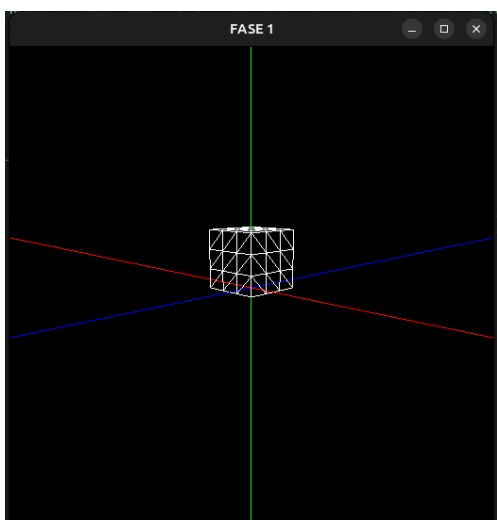


Figura 3 : Validação de translação

isolada (Cubo).

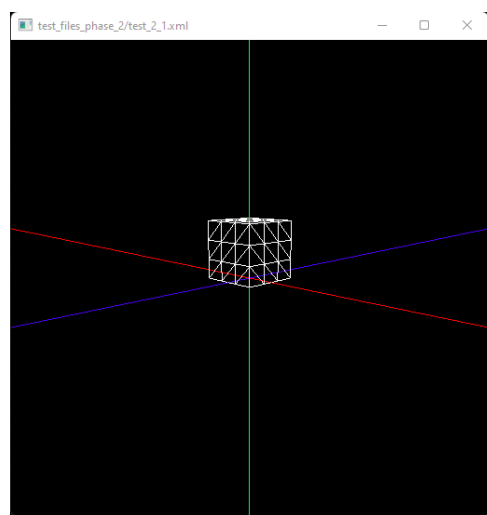


Figura 4 : Cubo (Exemplo)

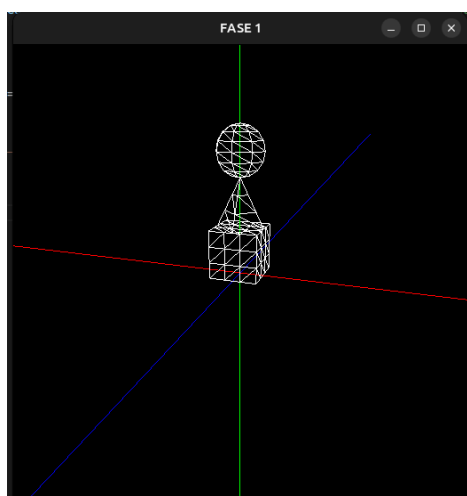


Figura 5 : Teste de grupos aninhados

e herança de transformações.

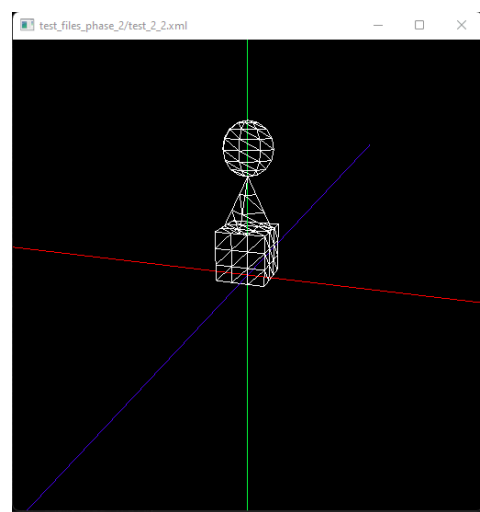


Figura 6 : Conjunto de Figuras (Exemplo)

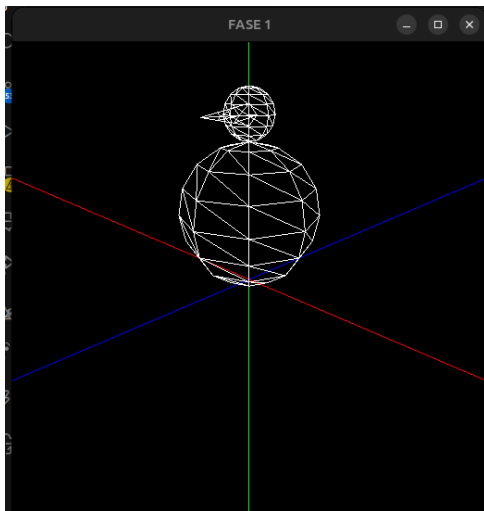


Figura 7 : Renderização do modelo

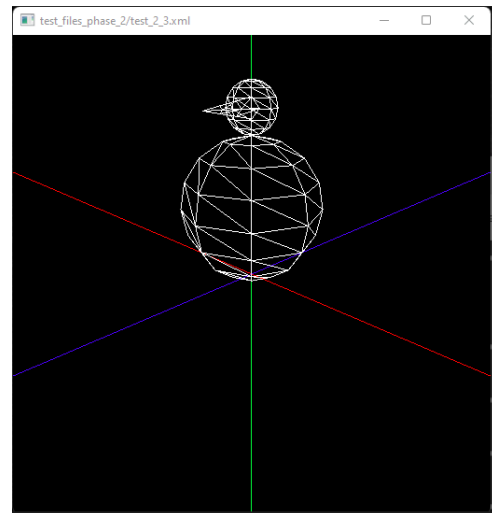


Figura 8 : Boneco de Neve (Exemplo)

articulado (Boneco de Neve).

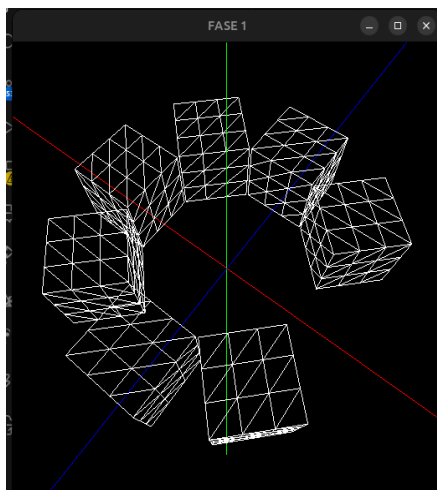


Figura 9 : Anel de cubos validando

a acumulação TRS.

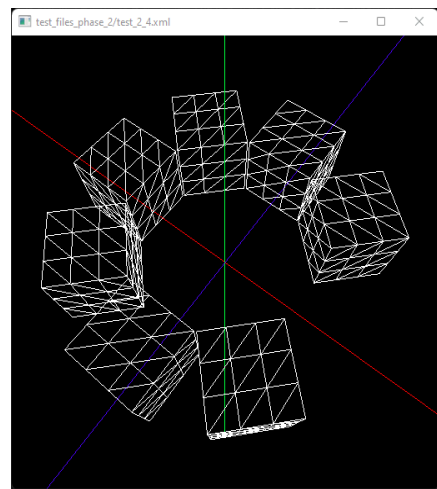


Figura 10 : Anel de cubos (Exemplo)

6. Conclusão e Trabalho Futuro

A conclusão sintetiza o sucesso da Fase 2 e prepara o caminho para a próxima etapa.

- **Conclusão:** O motor evoluiu com sucesso de um visualizador estático para um sistema baseado em **Scene Graph**. A implementação de um parser recursivo e a gestão da **Matrix Stack** do OpenGL foram os marcos principais desta fase.
- **Trabalho Futuro (Fase 3):**
 1. **Animação:** Substituir os ângulos estáticos por rotações e translações dependentes do tempo.
 2. **Curvas de Catmull-Rom:** Implementar trajetórias suaves para os planetas e cometas.
 3. **VBOs:** Otimizar a renderização, passando do *Immediate Mode* (glBegin/glEnd) para *Vertex Buffer Objects* para melhorar a performance.

7. Referências Bibliográficas

Deve listar as fontes técnicas utilizadas:

1. **OpenGL Reference:** "glPushMatrix, glPopMatrix and Transformation Matrices." [Online]. Disponível em: <https://www.opengl.org/>.
2. **TinyXML-2:** "The TinyXML-2 Library for C++ Parsing." [Online]. Disponível em: <https://github.com/leethomason/tinyxml2>.
3. **GLUT:** "OpenGL Utility Toolkit Reference."